

Experiment 2

Client–Server Communication Between Two Virtual Machines

Objective

- Create/prepare two Ubuntu Virtual Machines in VirtualBox.
- Configure both VMs on the same virtual network.
- Assign/check the IP address of both VMs.
- Test connectivity using the ping command.
- Designate one VM as the Server.
- Designate the other VM as the Client.
- Create a TCP server program on the Server VM.
- Create a TCP client program on the Client VM.
- Establish a connection between the two VMs.
- Send and receive messages through the chat application.
- Demonstrate the working chat application.

Theory

TCP (Transmission Control Protocol) provides a reliable, connection-oriented communication channel between two hosts on a network. A TCP socket program typically involves a server that binds to an IP address and port, listens for incoming connections, and accepts a client's request. Once the connection is established, both sides can exchange messages using `send()` and `recv()` calls until either side closes the connection. In this experiment, two Ubuntu virtual machines running in VirtualBox are connected on the same internal network and configured with static IP addresses, and Python's built-in socket module is used to implement a simple two-way chat application between them.

Network Setup and IP Configuration

Both VMs ("Ubuntu Lab 1" acting as the Server, and "Client") were placed on the same VirtualBox internal/host-only network. The **ip addr** command was used on each VM to inspect the existing interfaces, after which a static IP address was assigned to the `enp0s3` interface on each machine using **sudo ip addr add <ip>/24 dev enp0s3**.

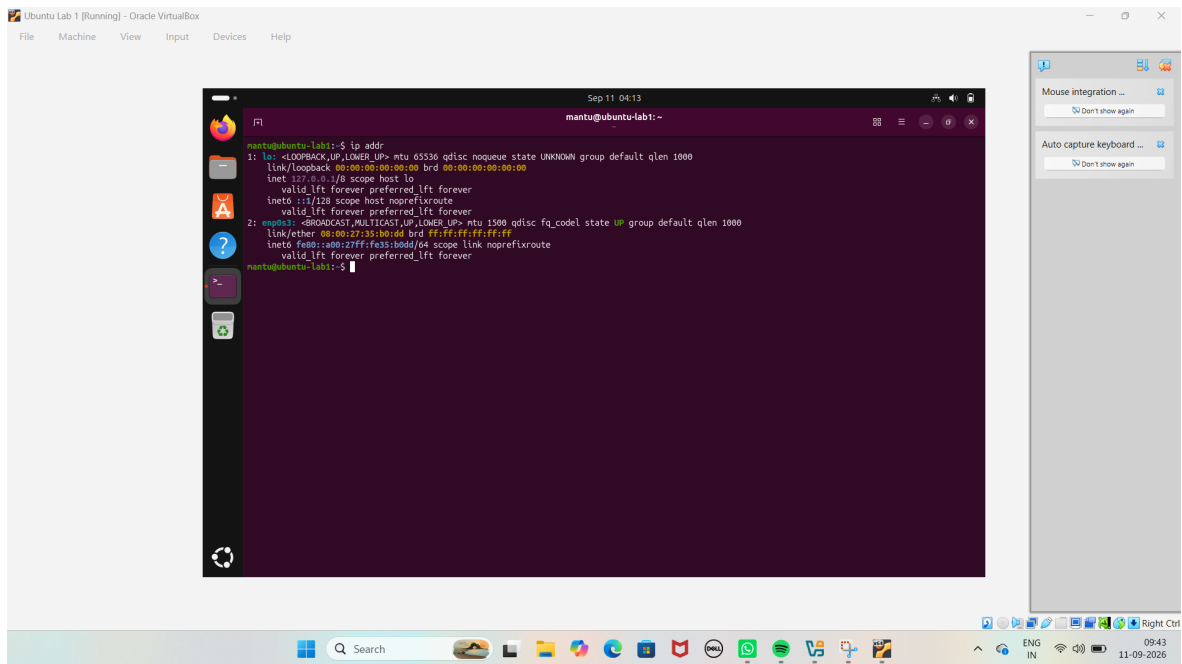


Fig. 1 — Initial network interface check on the Server VM (ip addr) before a static IP is assigned.

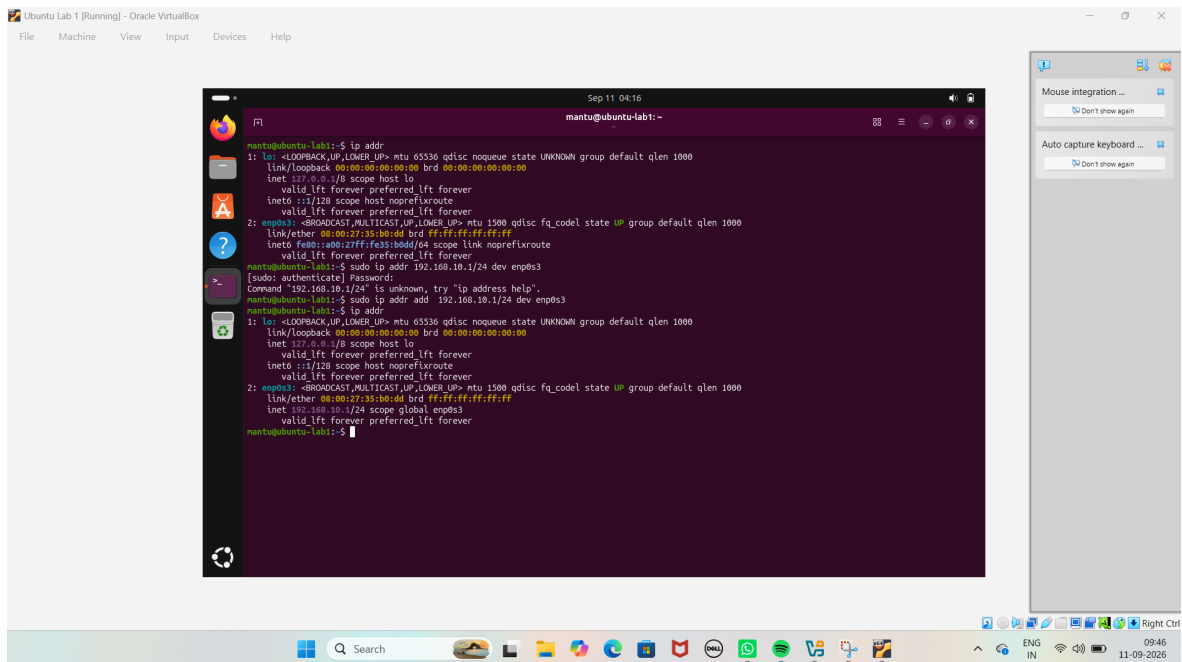


Fig. 2 — Static IP 192.168.10.1/24 assigned to the Server VM's enp0s3 interface.

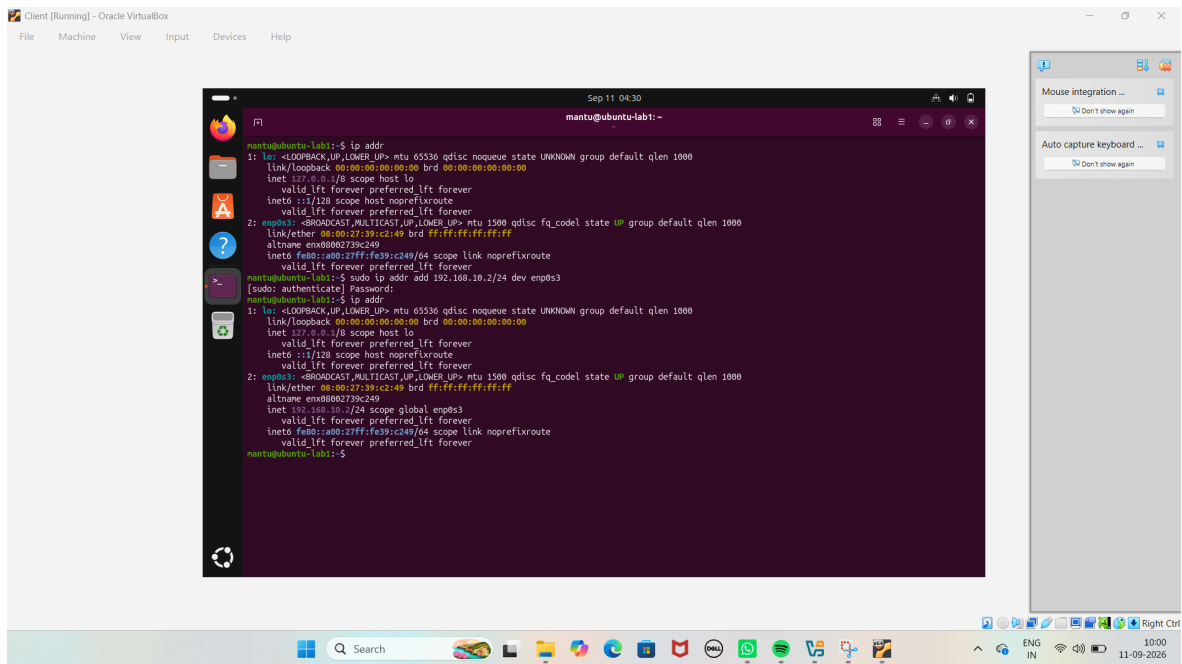


Fig. 3 — Static IP 192.168.10.2/24 assigned to the Client VM's enp0s3 interface.

With both interfaces configured, connectivity between the two VMs was verified using the **ping** command from each VM to the other's IP address before proceeding to the chat application, confirming that the two machines could reach each other over the virtual network.

TCP Server Program (server.py)

The server program runs on the VM assigned the IP address 192.168.10.1. It creates a TCP socket, binds it to the host IP and port 5000, listens for an incoming connection, and then enters a loop that alternately receives a message from the client and sends a reply back, until either side sends “exit”.

```
import socket

HOST = "192.168.10.1"
PORT = 5000

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)

print(f"Server started on {HOST}:{PORT}")
print("Waiting for client...")

conn, addr = server_socket.accept()
print(f"Connected by {addr}")

while True:
    message = conn.recv(1024).decode()
    if not message or message.lower() == "exit":
        print("Client disconnected.")
        break
    print("Client:", message)
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break

conn.close()
server_socket.close()
```

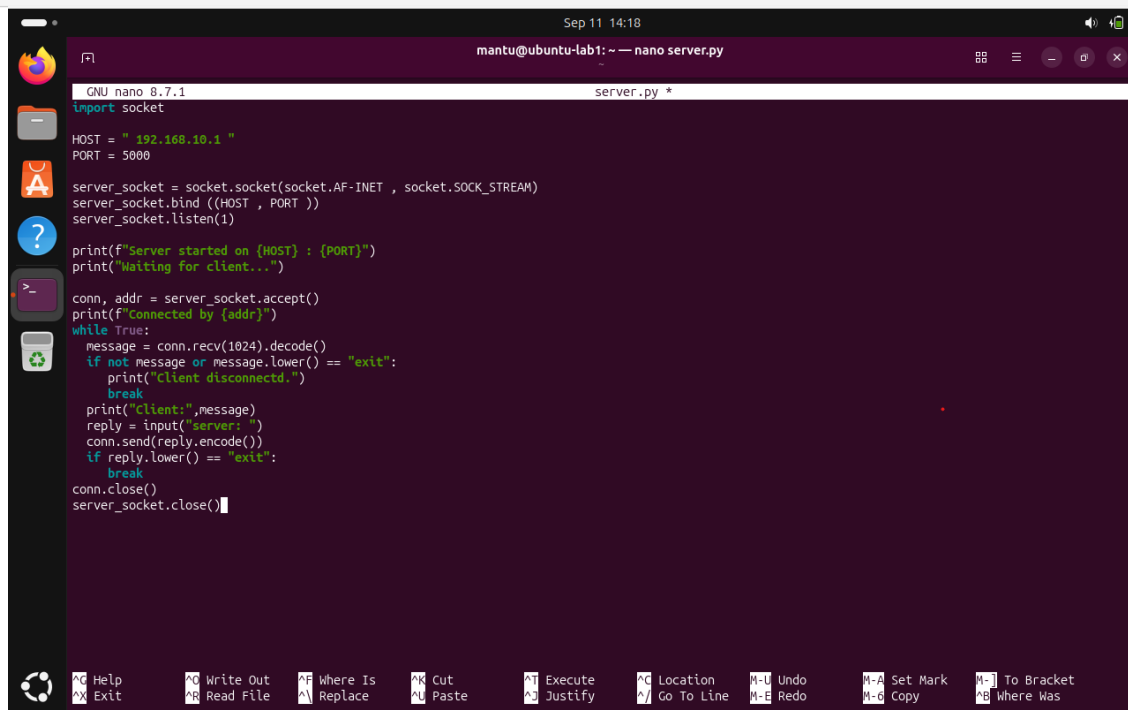


Fig. 4 — server.py being written on the Server VM using the nano editor.

TCP Client Program (client.py)

The client program runs on the second VM (192.168.10.2). It creates a TCP socket and connects to the server's IP address and port, then enters a loop that alternately sends a message to the server and waits for the server's reply, until either side sends "exit".

```
import socket

server_ip = "192.168.10.1"
port = 5000

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((server_ip, port))
print(f"Connected to server {server_ip}:{port}")

while True:
    message = input("Client: ")
    client_socket.send(message.encode())
    if message.lower() == "exit":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)
    if reply.lower() == "exit":
        break

client_socket.close()
```

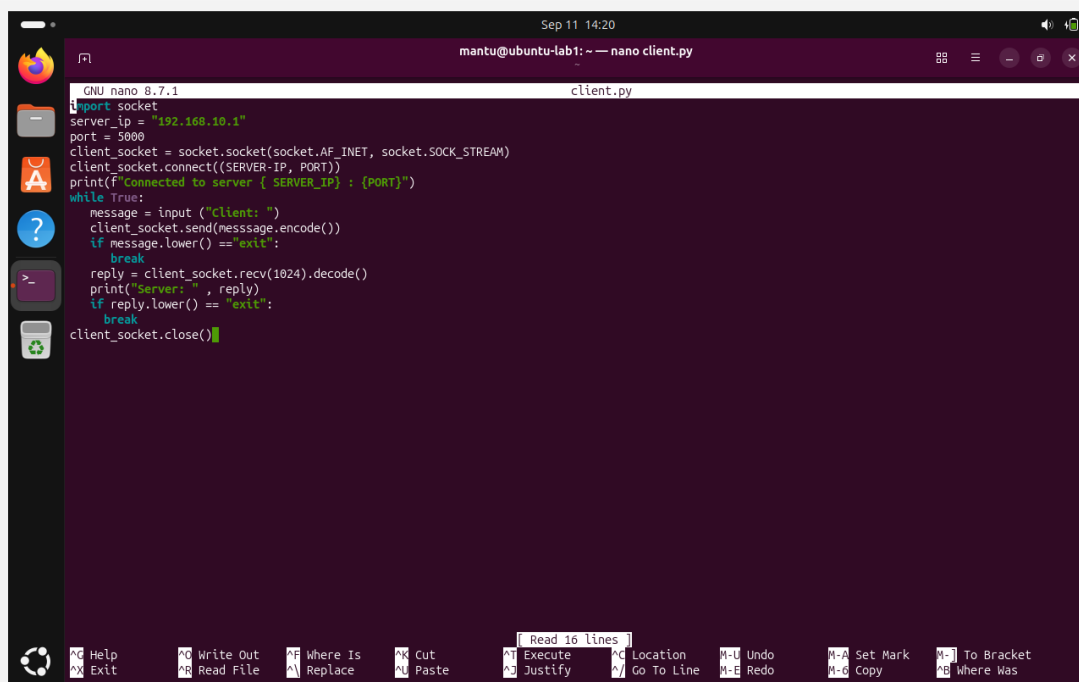
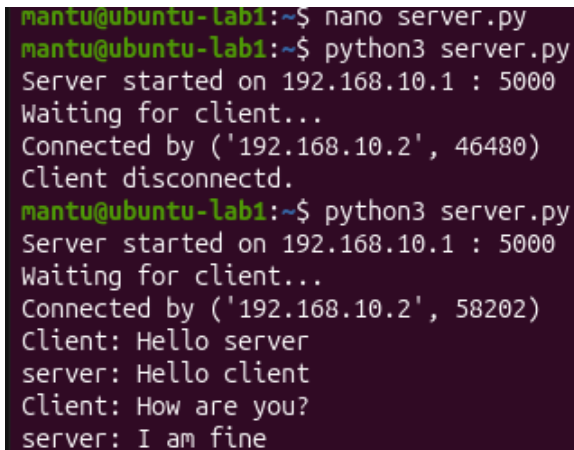


Fig. 5 — client.py being written on the Client VM using the nano editor.

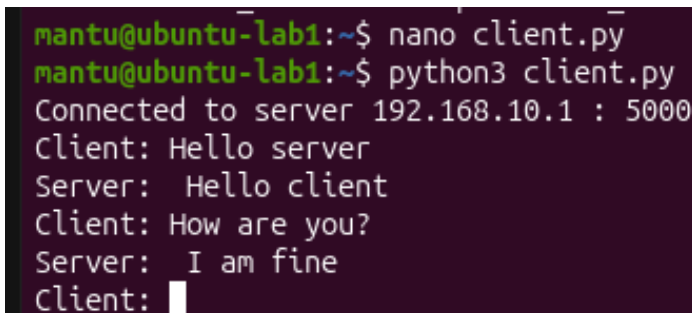
Execution and Output

The server program was started first on the Server VM with **python3 server.py**, which began listening on 192.168.10.1:5000. The client program was then started on the Client VM with **python3 client.py**, which connected to the server. Messages were then exchanged interactively between the two terminals, demonstrating a working two-way TCP chat.



```
mantu@ubuntu-lab1:~$ nano server.py
mantu@ubuntu-lab1:~$ python3 server.py
Server started on 192.168.10.1 : 5000
Waiting for client...
Connected by ('192.168.10.2', 46480)
Client disconnectd.
mantu@ubuntu-lab1:~$ python3 server.py
Server started on 192.168.10.1 : 5000
Waiting for client...
Connected by ('192.168.10.2', 58202)
Client: Hello server
server: Hello client
Client: How are you?
server: I am fine
```

Fig. 6 — Server-side terminal: server started, client connected, and messages exchanged.



```
mantu@ubuntu-lab1:~$ nano client.py
mantu@ubuntu-lab1:~$ python3 client.py
Connected to server 192.168.10.1 : 5000
Client: Hello server
Server: Hello client
Client: How are you?
Server: I am fine
Client: 
```

Fig. 7 — Client-side terminal: connected to the server and exchanging chat messages.

Observation

- Both VMs were successfully configured on the same virtual network with static IP addresses 192.168.10.1 (Server) and 192.168.10.2 (Client).
- Connectivity between the two VMs was confirmed before running the chat application.
- The TCP server successfully accepted a connection from the client and the two machines exchanged messages in both directions.
- The chat session could be terminated cleanly by either side sending “exit”.

Conclusion

A basic TCP-based client–server chat application was successfully implemented and demonstrated between two separate Ubuntu virtual machines in VirtualBox. The experiment confirmed static IP configuration, network connectivity between VMs, and reliable two-way message exchange using Python sockets, reinforcing the fundamentals of socket programming and TCP-based communication.